Universidad Rafael Landivar, Quetzaltenango

Facultad de Ingeniería
MANEJO E IMPLEMENTACIÓN DE ARCHIVOS

ING. TORRES BARRIOS SERGIO RAUL

Martes 27 de octubre de 2026



“Laboratorio No. 2: Agenda de Contactos con Acceso Aleatorio ”

SOBERANIS GALINDO DAN MAHLI EST16589-25
NOGUERA GONZÁLEZ JOSÉ RODERICO EST16191-25
DE LEÓN GONZÁLEZ ALEJANDRO DANIEI EST15024-25

1. Estructura del Registro

El sistema utiliza una estructura de longitud fija de 100 bytes por cada contacto guardado en contactos.dat. Los campos se completan con espacios a la derecha utilizando el método ljust() dentro de la función de ayuda _rellenar. La distribución de los bytes es la siguiente:

- **id:** 6 bytes
- **nombre:** 30 bytes
- **telefono:** 15 bytes
- **correo:** 30 bytes
- **ciudad:** 18 bytes
- **Terminador de línea (\n):** 1 byte
- **Tamaño total del registro:** 100 bytes exactos.

2. Implementación del Acceso Aleatorio con `seek()` y `tell()`

El acceso aleatorio se logra calculando el desplazamiento (offset) exacto del registro en el archivo físico. Esta posición se obtiene multiplicando el número de registro por el tamaño fijo de 100 bytes (`pos * RECORD_SIZE`). La instrucción `seek(offset)` mueve el puntero directamente a ese byte específico antes de leer o escribir, mientras que `tell()` se invoca antes y después de la operación de E/S para auditar la posición exacta del cursor.

A continuación, los fragmentos relevantes extraídos de `contact_store.py`:

- Fragmento de lectura directa (`_read_record`):

```
451     # -- lectura / escritura de registros ----------------------------------
452     def _read_record(self, pos: int) -> tuple[Contact, AccessTrace]:
453         traza = AccessTrace("LEER", pos, pos * RECORD_SIZE)
454         with _guard(self.dat_path, "leer"):
455             with open(self.dat_path, "rb") as f:
456                 f.seek(traza.offset)            # Salto directo a la posición calculada
457                 traza.tell_antes = f.tell()     # Verifica dónde quedó el puntero
458                 raw = f.read(RECORD_SIZE)       # Lee exactamente 100 bytes
459                 traza.tell_despues = f.tell()   # Verifica el puntero tras la lectura
460         traza.bytes_io = len(raw)
461         return decode_record(raw, pos), traza
```

- Fragmento de modificación en el lugar (`update`):

```
501     def update(self, contacto: Contact) -> AccessTrace:
502         """Sobrescribe SOLO el registro del contacto; el resto no se reescribe."""
503         c = contacto.normalizado()
504         c.validar()
505         pos = self._index.get(c.id)
506         if pos is None or not self.dat_path.exists():
507             raise ContactNotFoundError(f"No existe ningún contacto con ID {c.id}.")
508         traza = AccessTrace("MODIFICAR", pos, pos * RECORD_SIZE)
509         raw = encode_record(c)
510         with _guard(self.dat_path, "escribir en"):
511             with open(self.dat_path, "r+b") as f:
512                 f.seek(traza.offset)
513                 actual = decode_record(f.read(RECORD_SIZE), pos)
514                 if actual.id != c.id:
515                     raise DataFileCorruptError(
516                         "El índice no coincide con el archivo; use Herramientas > "
517                         "Verificar integridad para reconstruirlo."
518                     )
519                 f.seek(traza.offset)            # Ubica el puntero en el inicio del registro
520                 traza.tell_antes = f.tell()
521                 f.write(raw)                    # Sobrescribe solo los 100 bytes de este registro
522                 f.flush()
523                 os.fsync(f.fileno())            # Fuerza la escritura en disco
524                 traza.tell_despues = f.tell()
525         traza.bytes_io = len(raw)
526         self.seal()
527         logger.info("Contacto modificado: id=%s registro=%s", c.id, pos)
528         return traza
```

3. Funcionamiento del Índice y Eficiencia de Búsqueda

El índice funciona como un mapa o tabla hash cargada en la memoria RAM. Físicamente, se almacena en indice.json como un diccionario que vincula la clave primaria (el id del contacto) con su número de registro correspondiente (su posición secuencial desde 0).

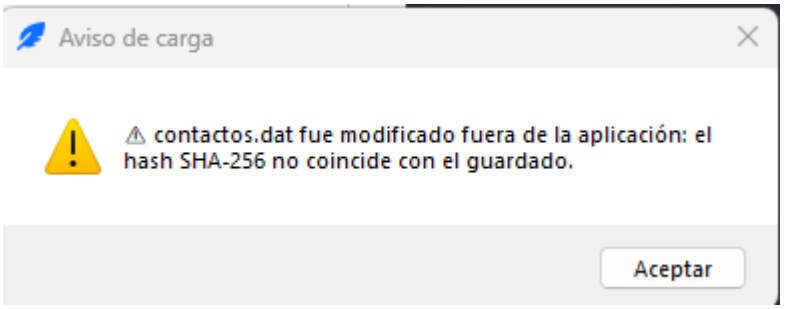
Cuando el usuario busca un contacto por su ID, la aplicación consulta este diccionario, recupera el número de registro en tiempo constante (complejidad O(1)), multiplica por 100 bytes y usa seek() para leer directamente ese único fragmento en el disco duro.

Por qué es más eficiente:

En una búsqueda secuencial clásica (complejidad O(N)), el programa se ve forzado a leer el archivo de texto línea por línea (registro por registro), cargando múltiples bloques de datos desde el disco duro a la memoria hasta encontrar el ID que coincida. Este proceso multiplica drásticamente las costosas operaciones de Entrada/Salida (I/O). Al utilizar un índice primario combinado con registros de tamaño fijo, la búsqueda se vuelve una operación matemática directa (offset), garantizando una única lectura rápida en el disco independientemente de si hay 100 o 100,000 contactos guardados.

4. Evidencia (capturas) de la verificación de integridad detectando un archivo alterado manualmente.

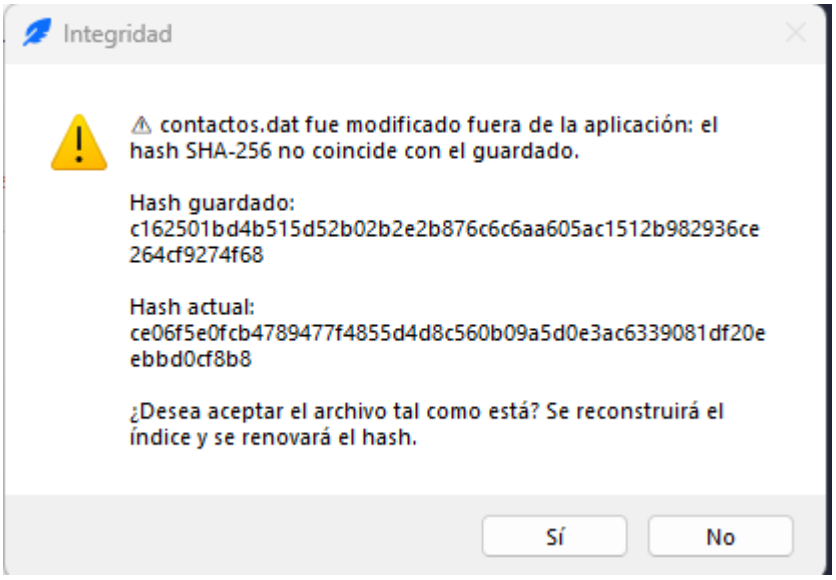
Al iniciar el programa cuando los datos fueron modificados de una forma externa o más bien manual, el programa lo detecta automáticamente y lanza un aviso de esto antes de seguir con su funcionamiento habitual:



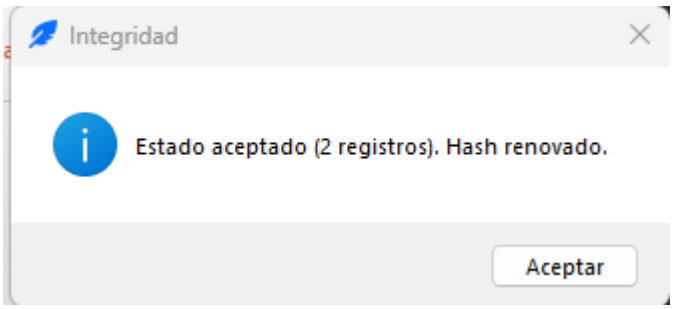
Si se ignora este aviso y se continúa con la aplicación, se mostrará constantemente un recordatorio de esto:



Al utilizar la herramienta de verificación de integridad, se mostrará esta ventana en donde se notifica al usuario de lo que está ocurriendo y se le preguntará si acepta o rechaza continuar trabajando con estos datos.

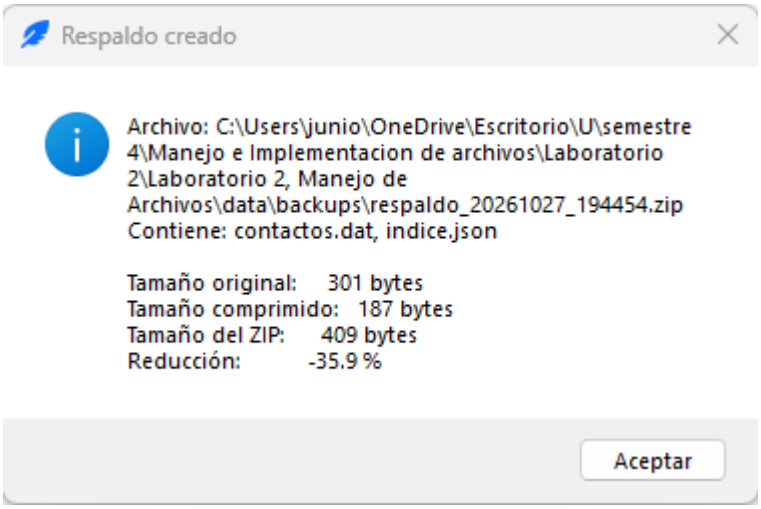


En caso de aceptarlo, se hará un re-hashing y se continuará con el funcionamiento normal de la aplicación.


Integridad
Estado aceptado (2 registros). Hash renovado.
Aceptar

5. Comparación del tamaño del archivo original contra el respaldo ZIP.

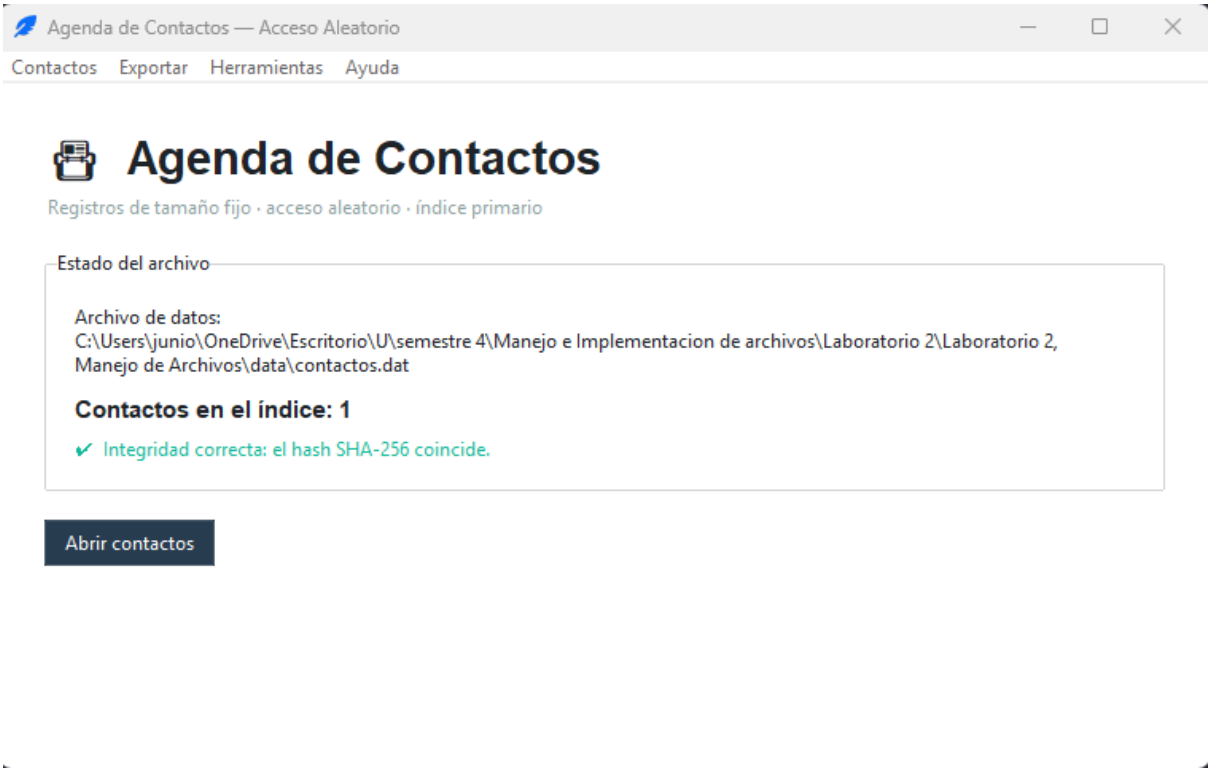
Tomando como ejemplo una prueba realizada, el mismo programa nos muestra la comparación entre el tamaño original del archivo y el tamaño final del respaldo ZIP.



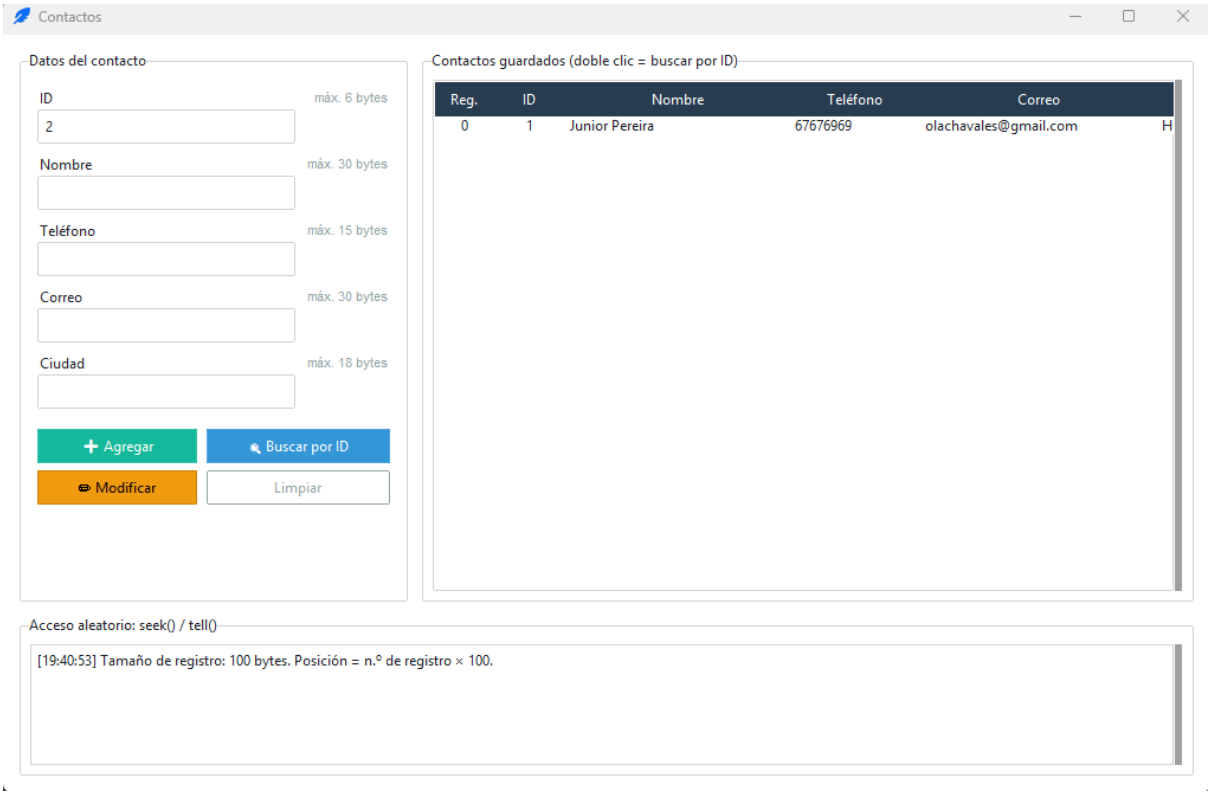
En este caso en particular, al ser el código tan pequeño (301 bytes), generar un archivo en formato ZIP resulta contraproducente y el resultado en un mayor peso que el archivo original.

6. Capturas de pantalla del funcionamiento de la aplicación.

La aplicación se basa en una interfaz gráfica simple e intuitiva, en donde al correr la aplicación, se ve lo siguiente:

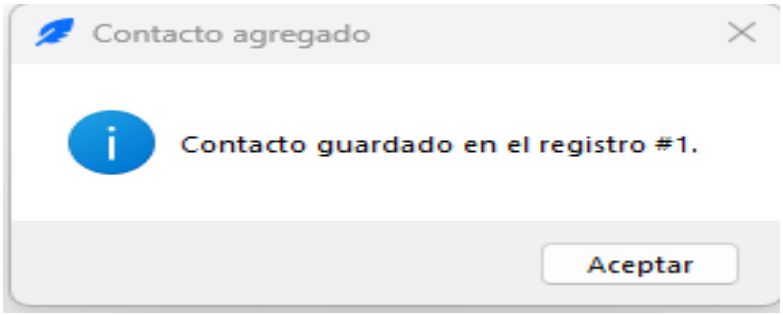


Se muestra una agenda de contactos y ubicación de los archivos. Al presionar el botón principal “Añadir contactos” se despliega la siguiente ventana:

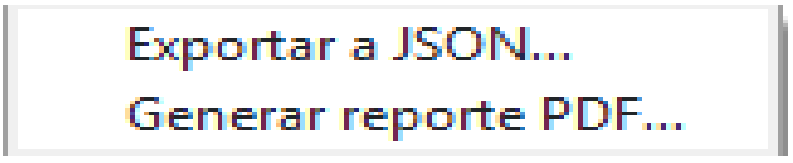


Donde se muestran los contactos registrados en el archivo, permitiendo: Agregar, Buscar por ID, Modificar y Limpiar.

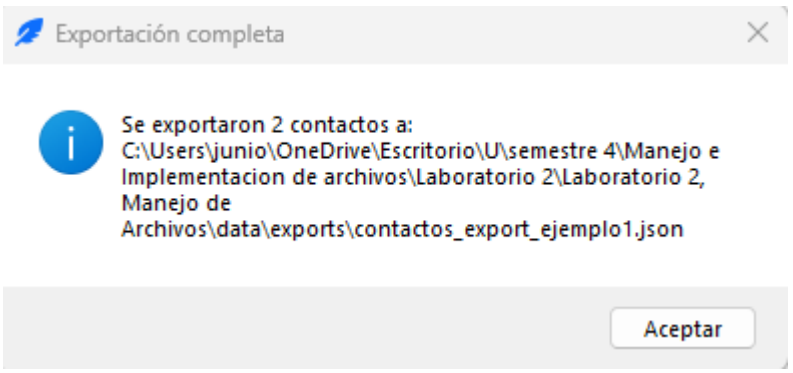
Para agregar, se debe de llenar los campos y presionar el botón verde “Agregar”. Hecho correctamente se muestra la siguiente ventana de confirmación:



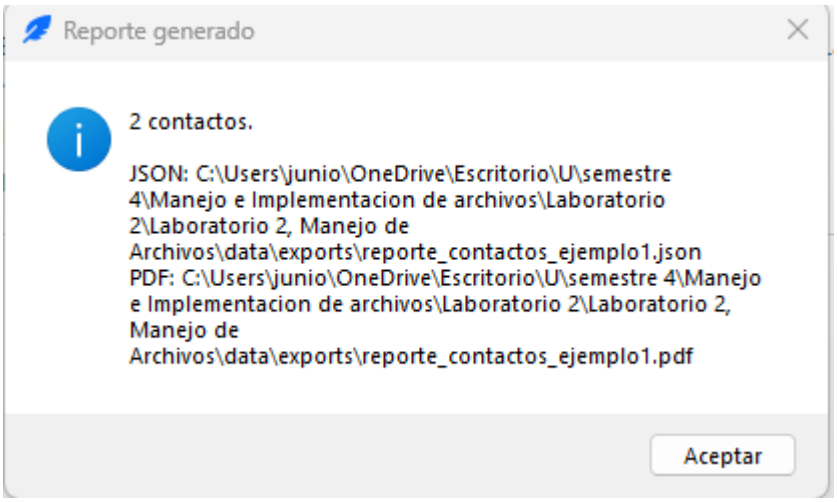
Cuando vamos al apartado de exportar, se ven las siguientes opciones:



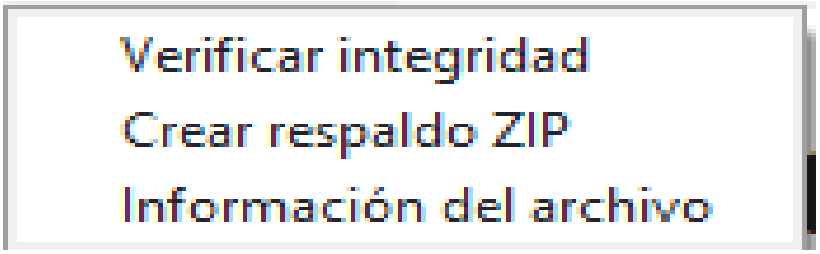
En donde al exportar JSON, después de escoger la ubicación donde se guardará el archivo, se ve la siguiente confirmación:



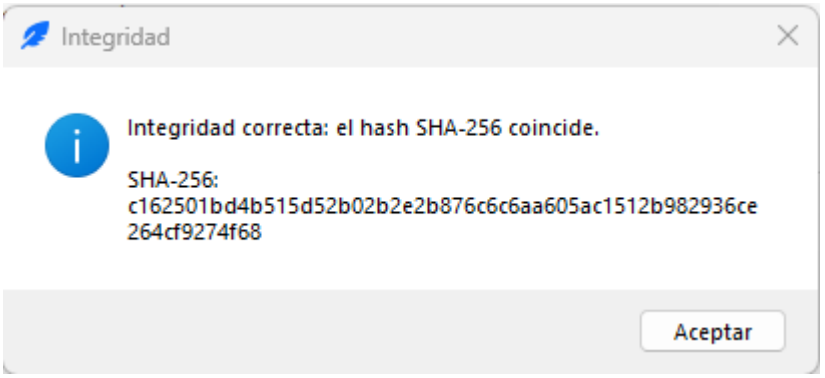
Al generar reporte (generado en formato PDF), después de escoger la ubicación donde se guardará el archivo, se ve la siguiente confirmación:



El apartado de “Herramientas” despliega las siguientes opciones:



Verificar integridad, muestra una ventana emergente con información de los archivos:



Crear respaldo ZIP, genera automáticamente (en una carpeta ya hecha y ubicada en los archivos de la aplicación) un respaldo de los datos en formato ZIP, con información como ubicación, tamaño original, tamaño en formato ZIP y porcentaje de reducción:

